

Somaiya Vidyavihar University
K. J. Somaiya School of Engineering, Mumbai-77

Batch: C3 Roll No.: 16010123325

Experiment / Assignment / Tutorial No. _5_

Grade: AA / AB / BB / BC / CC / CD / DD

Title: Analysis of sample vulnerable web applications for SQL injection and XSS using Burp Suite

Objective: Exploration of vulnerabilities for web based applications through XSS attack using DVWA & Burp Suite.

Expected Outcome of Experiment:

Course Outcome	After successful completion of the course students should be able to
	1. Understand Cross-site scripting (XSS) & SQLi. 2. Understand concept of DVWA & Burp Suite.

Books/ Journals/ Websites referred:

- OWASP Top Ten
- Damn Vulnerable Web Application (DVWA) Official Documentation
- Burp Suite Documentation
- PortSwigger Web Security Academy
- W. Stallings – Cryptography and Network Security
- NIST Secure Coding Guidelines

Pre Lab/ Prior Concepts:

Basics of web applications (client-server model, HTTP/HTTPS)

Understanding of cookies and sessions

Fundamentals of databases and SQL queries

HTML and JavaScript basics

Concept of input validation and sanitization

Somaiya Vidyavihar University

K. J. Somaiya School of Engineering, Mumbai-77

New Concepts to be learned:

- Cross-Site Scripting (XSS) types: Stored, Reflected, DOM-based
 - SQL Injection (SQLi) attack mechanisms
 - Web vulnerability testing using DVWA
 - Intercepting and modifying requests using Burp Suite
 - Secure coding practices and prevention techniques
-

Abstract:

This experiment focuses on identifying and understanding common web application vulnerabilities such as Cross-Site Scripting (XSS) and SQL Injection (SQLi). Using controlled environments like DVWA and tools such as Burp Suite, various attack scenarios are explored conceptually, including session hijacking, phishing, and unauthorized actions. The experiment emphasizes both exploitation awareness and the implementation of prevention techniques to build secure web applications.

Related Theory:

1. Cross-Site Scripting (XSS):

XSS is a client-side attack where malicious scripts are injected into trusted web applications.

Types:

- **Stored XSS:** Script stored in database and executed for all users
- **Reflected XSS:** Script reflected via URL/request
- **DOM-based XSS:** Manipulation happens in browser DOM

Conceptual Attack Scenarios:

- **Session Hijacking:** Stealing session cookies to impersonate users
- **Unauthorized Actions:** Executing actions on behalf of a victim
- **Phishing:** Fake forms to collect credentials
- **Keylogging:** Capturing keystrokes via injected scripts
- **Data Theft:** Extracting sensitive user information

2. SQL Injection (SQLi):

SQLi is a server-side attack where malicious SQL queries manipulate the database.

Somaiya Vidyavihar University
K. J. Somaiya School of Engineering, Mumbai-77

Conceptual Examples:

- **Retrieving Hidden Data:** Accessing restricted database records
- **Subverting Application Logic:** Bypassing authentication
- **UNION Attacks:** Combining queries to extract data from other tables
- **Database Enumeration:** Identifying schema, tables, columns
- **Blind SQL Injection:** Inferring data when no direct output is shown

3. DVWA (Damn Vulnerable Web Application):

- A deliberately insecure web application used for learning security concepts
- Allows testing at different difficulty levels (low → high)
- Provides practical exposure to vulnerabilities in a safe environment

4. Burp Suite:

- A web security testing tool used to:
- Intercept HTTP requests
 - Modify parameters
 - Analyze vulnerabilities
- Includes modules like Proxy, Repeater, Intruder

5. Prevention Mechanisms:**For XSS:**

- Input validation and output encoding
- Use of Content Security Policy (CSP)
- Escaping user inputs
- HTTPOnly cookies

For SQLi:

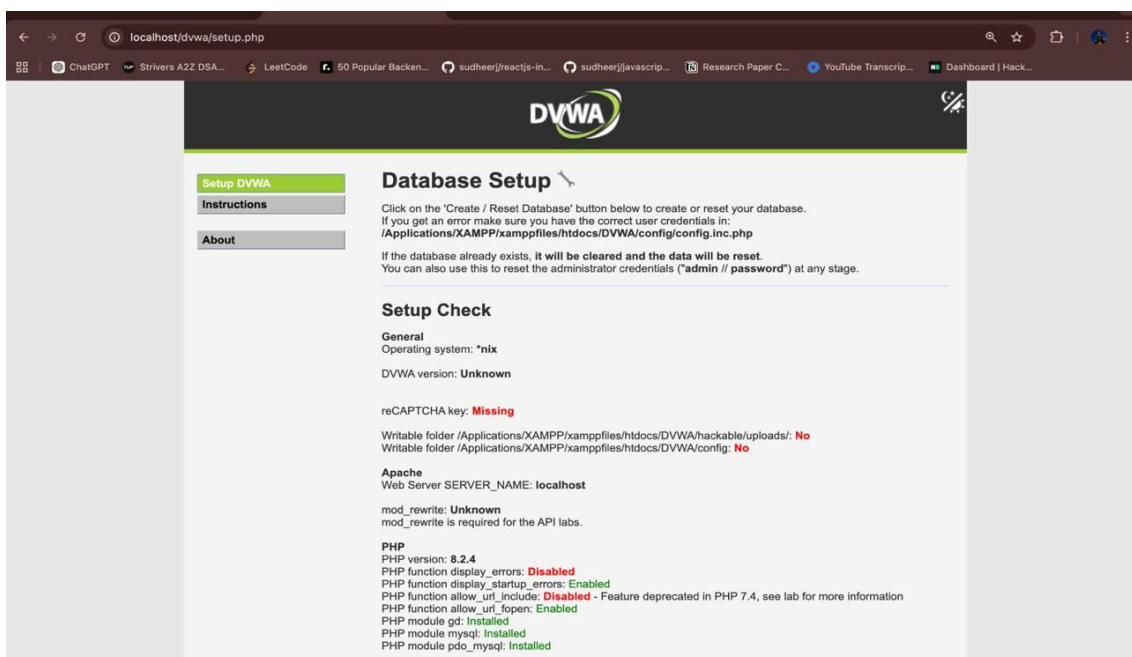
- Use of prepared statements / parameterized queries
- Input sanitization
- ORM frameworks
- Least privilege database access

Somaiya Vidyavihar University

K. J. Somaiya School of Engineering, Mumbai-77

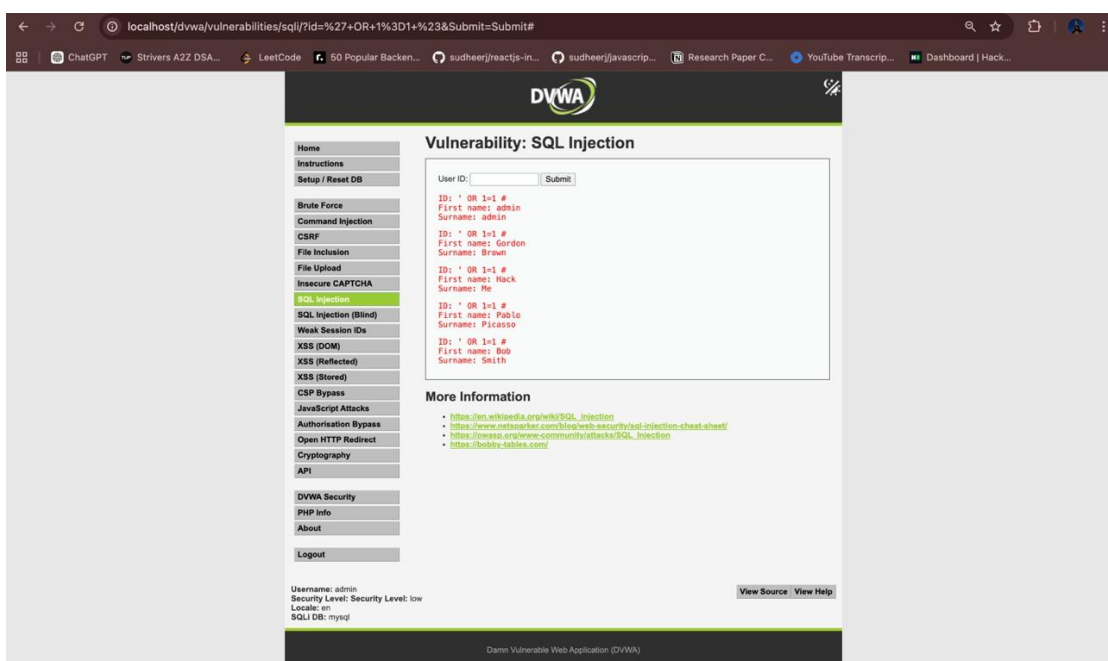
Implementation Details:

1: DVWA XSS Module Interface – Web application page used to test cross-site scripting vulnerabilities.



2: SQL Injection – Basic Payload Execution

SQL Injection using ' OR '1'='1 – Extracting multiple user records.

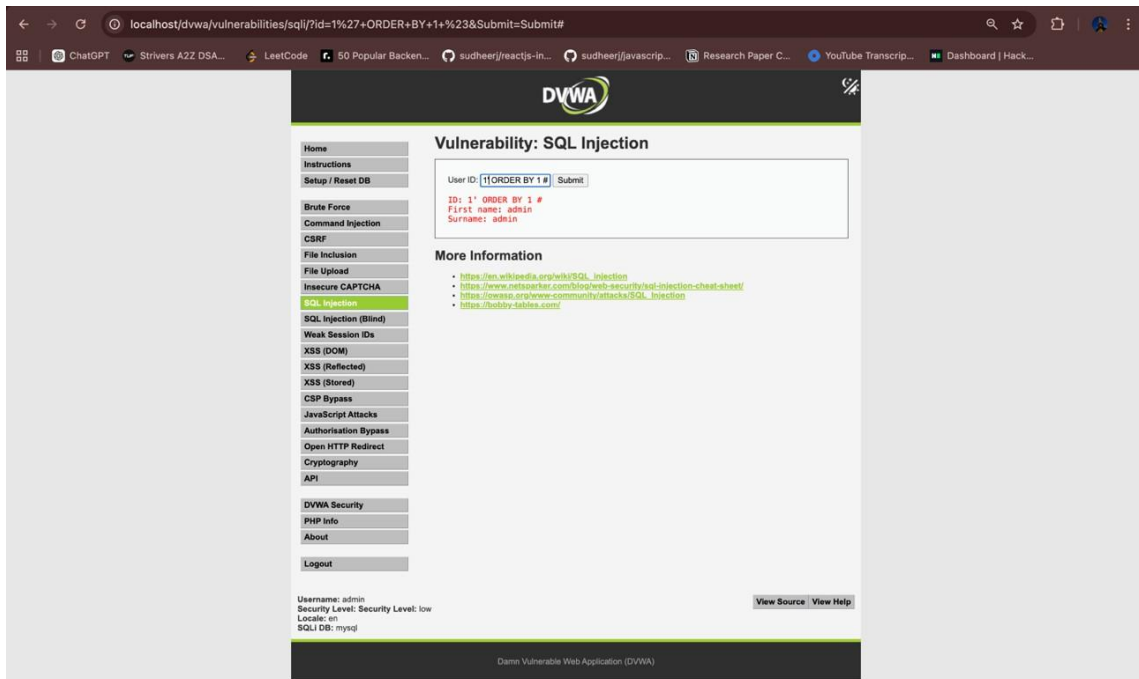


Somaiya Vidyavihar University

K. J. Somaiya School of Engineering, Mumbai-77

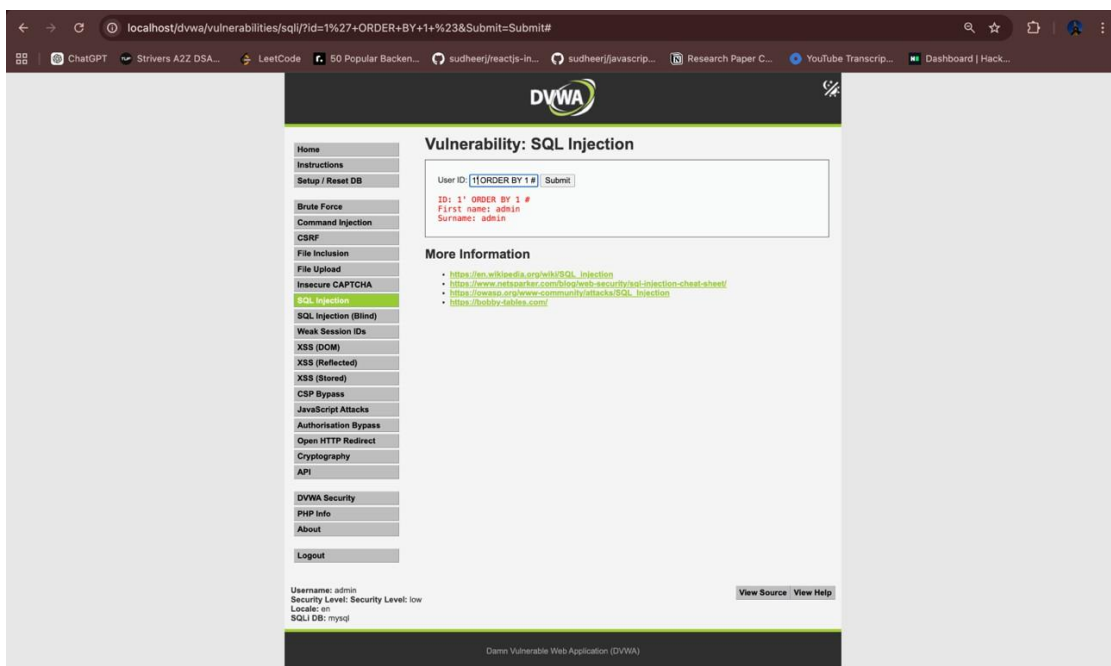
3: SQL Injection – Admin Login Bypass

Authentication Bypass using SQL Injection (admin' OR '1'=1)



4: SQL Injection – ORDER BY Enumeration

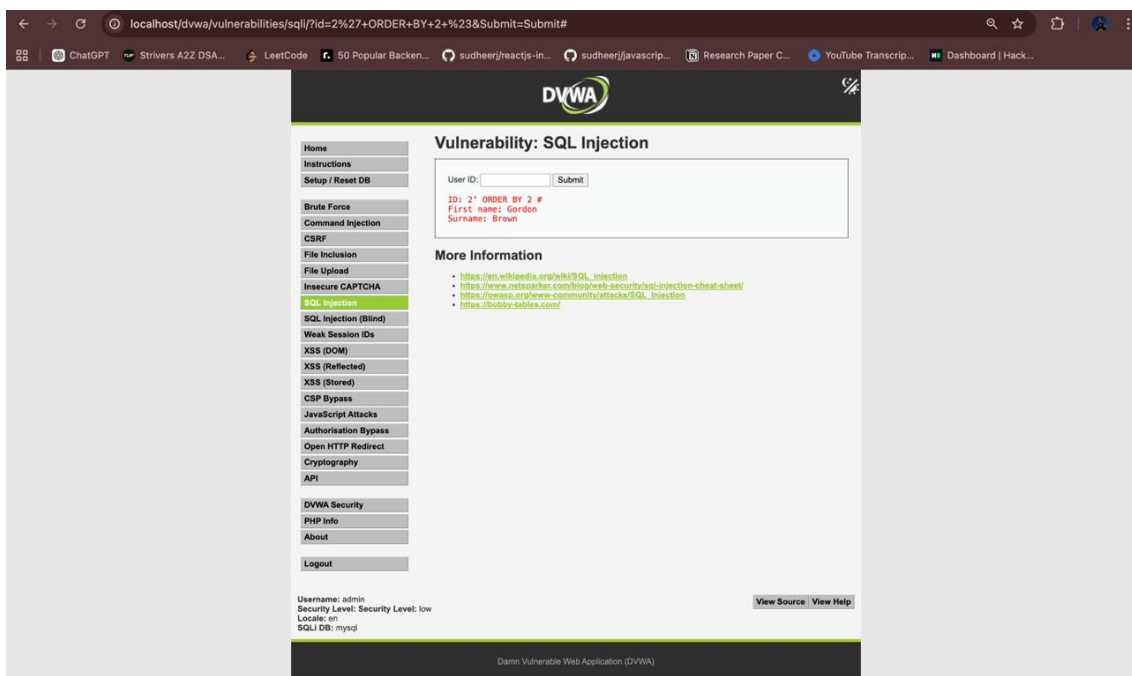
Column Enumeration using ORDER BY clause.



Somaiya Vidyavihar University K. J. Somaiya School of Engineering, Mumbai-77

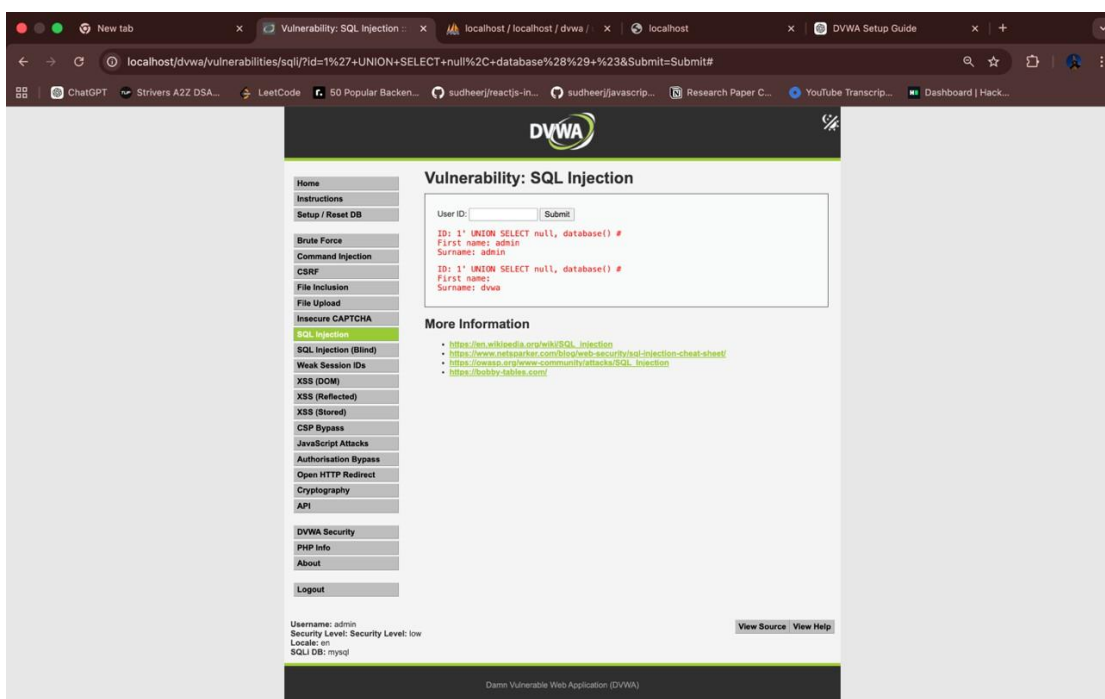
5: SQL Injection – ORDER BY (Next Column)

Determining number of columns using ORDER BY.



6: SQL Injection – Database Name Extraction

Extracting database name using UNION SELECT.

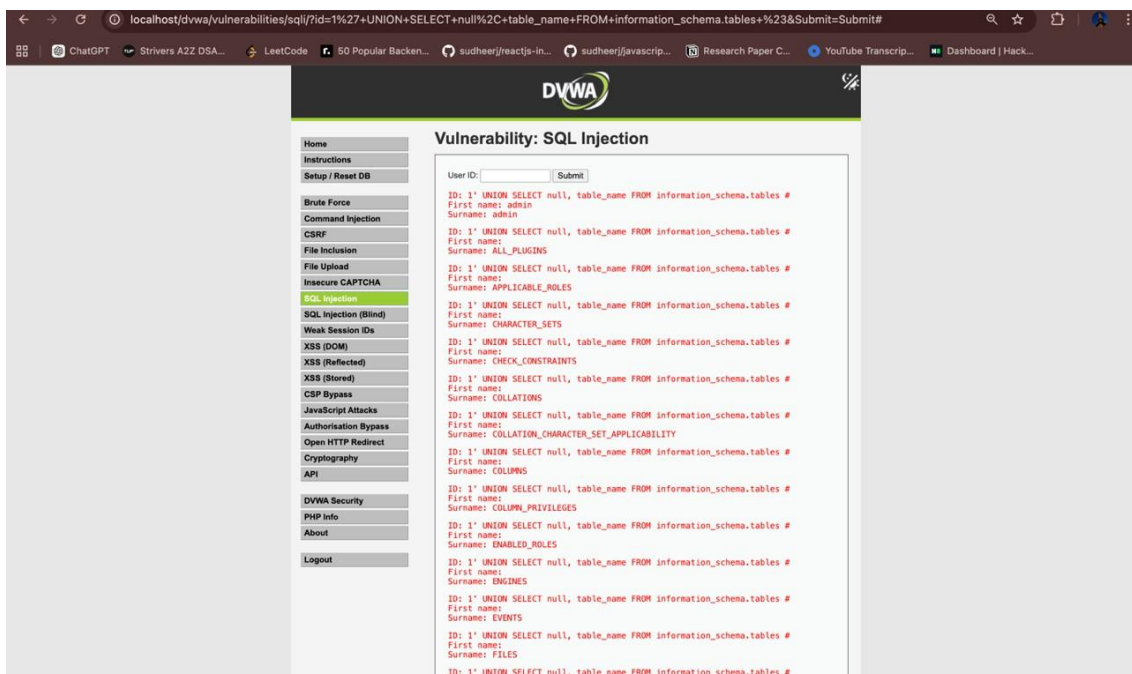


Somaiya Vidyavihar University

K. J. Somaiya School of Engineering, Mumbai-77

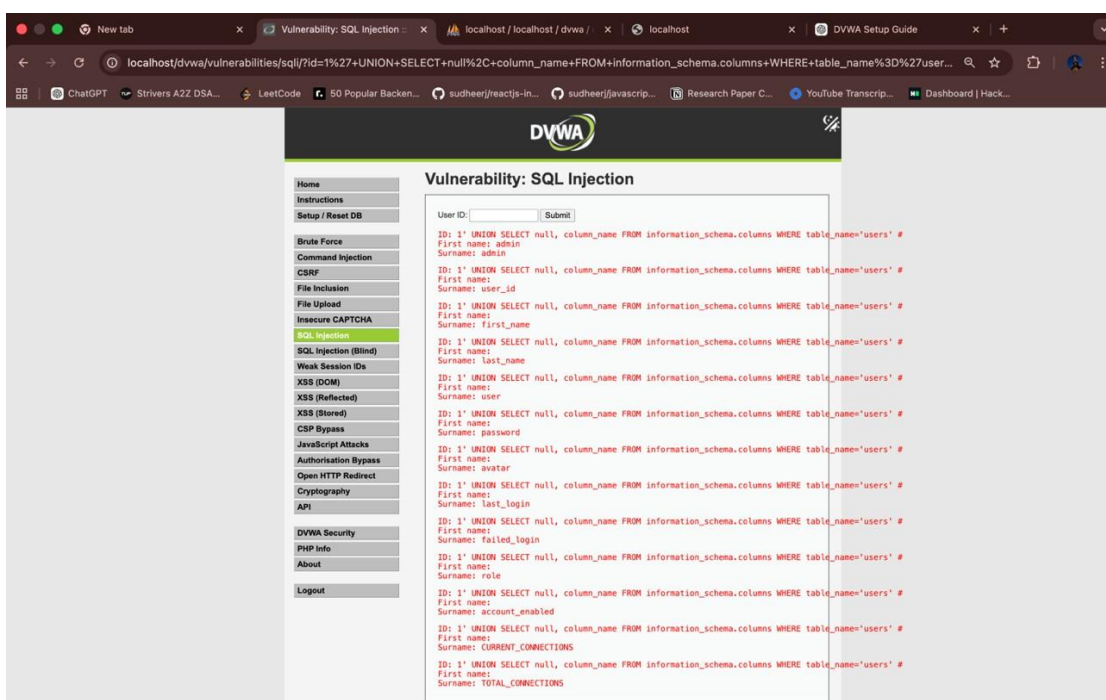
7: SQL Injection – Table Enumeration

Retrieving table names from information_schema.



8: SQL Injection – Column Enumeration

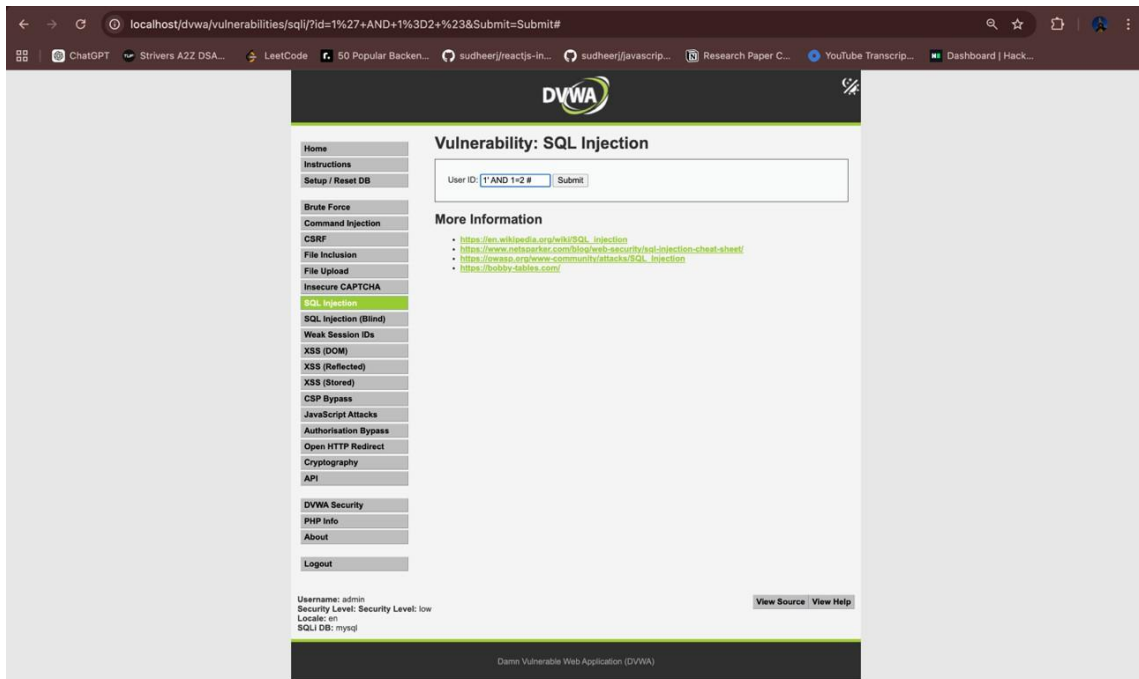
Extracting column names from users table.



Somaiya Vidyavihar University K. J. Somaiya School of Engineering, Mumbai-77

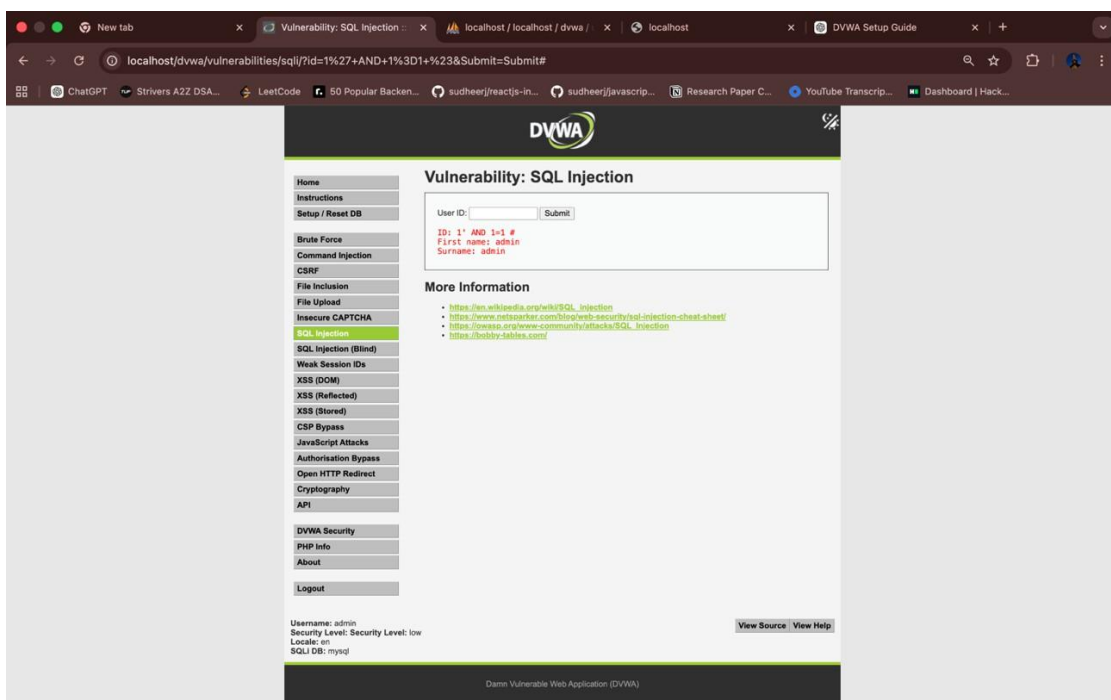
9: SQL Injection – False Condition

Testing false condition using AND operator.



10: SQL Injection – True Condition

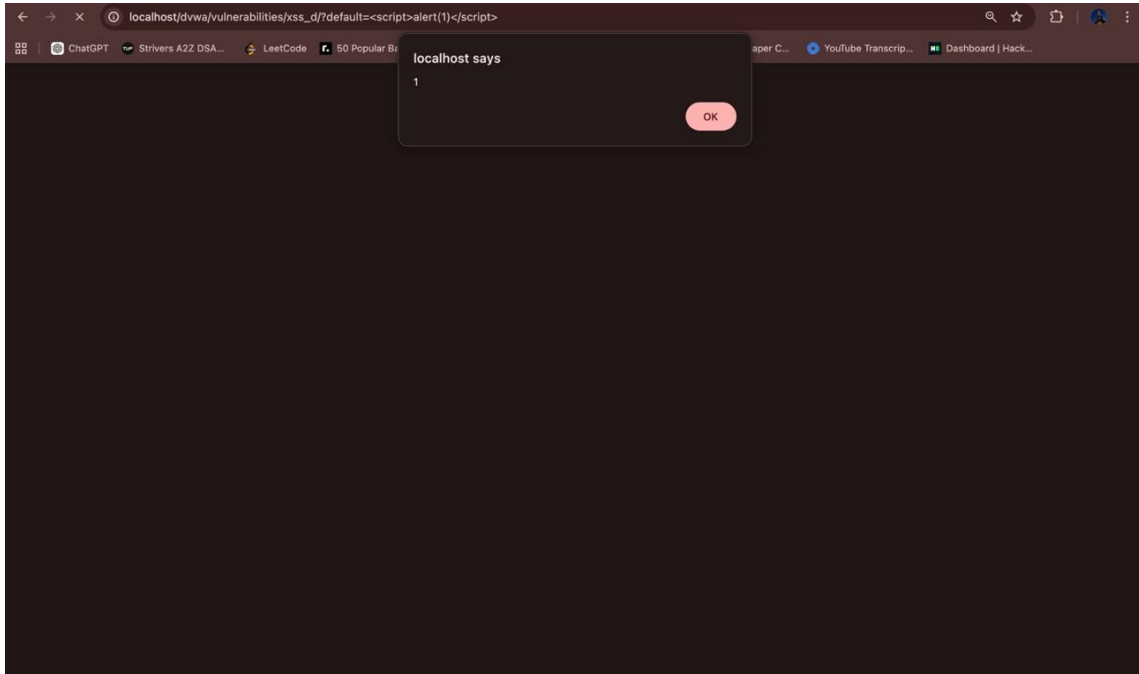
Validating injection using TRUE condition.



Somaiya Vidyavihar University K. J. Somaiya School of Engineering, Mumbai-77

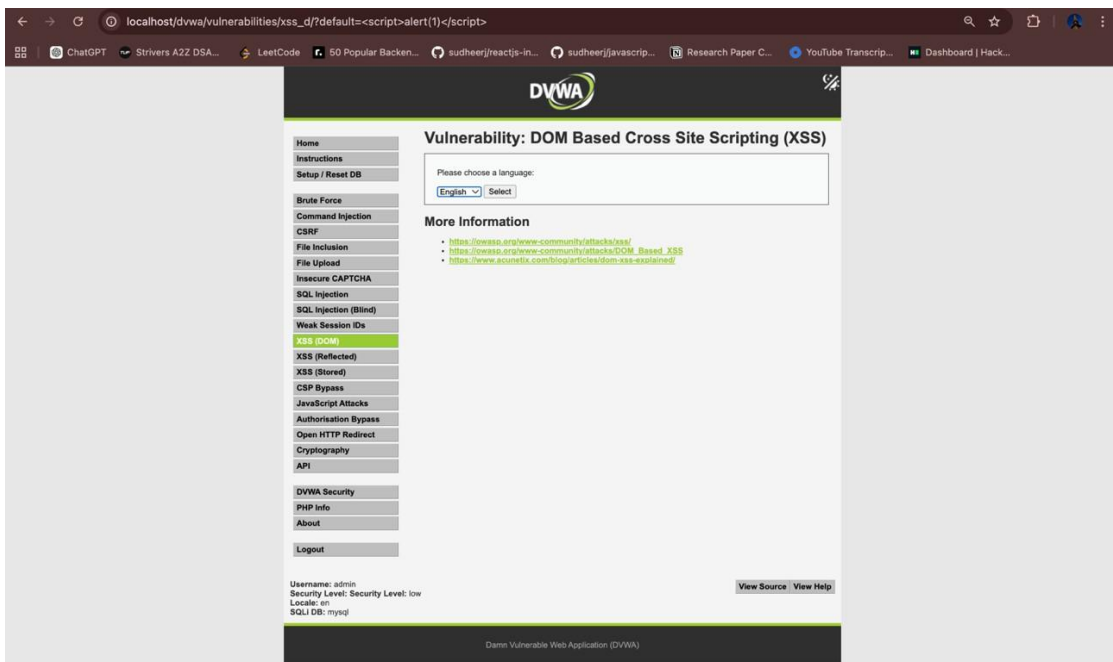
11: DOM-Based XSS Alert

DOM-Based XSS Execution using JavaScript payload `<script>alert(1)</script>`



12: DOM XSS Vulnerable Page

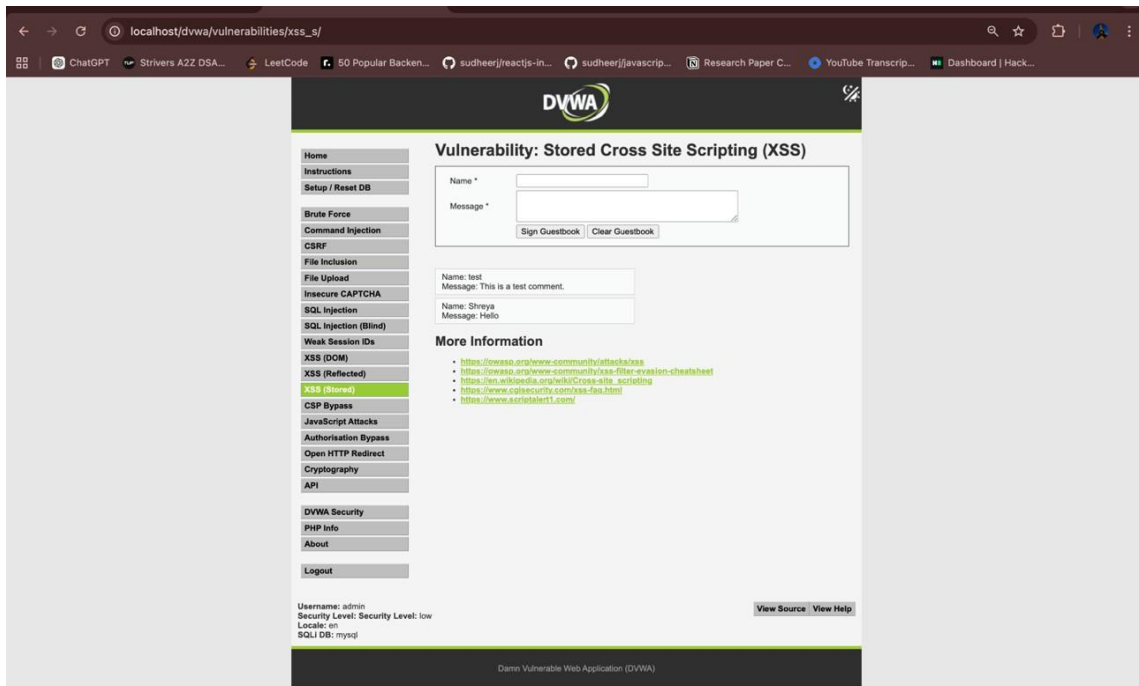
DVWA DOM-Based XSS module interface.



Somaiya Vidyavihar University K. J. Somaiya School of Engineering, Mumbai-77

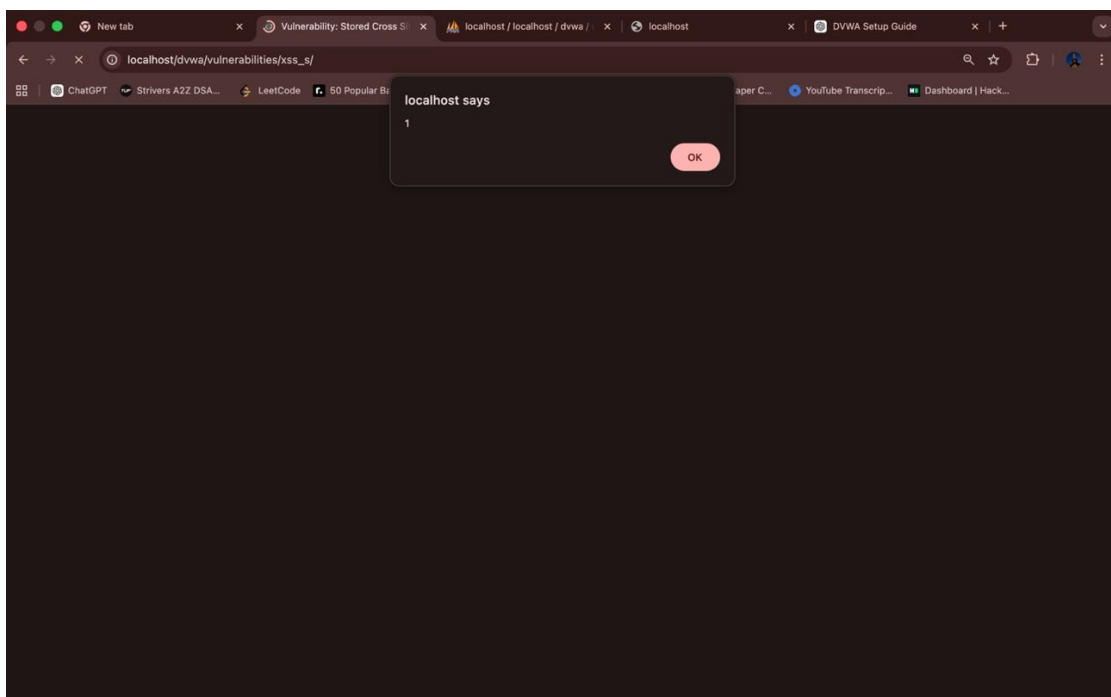
13: Stored XSS Input Page

Stored XSS – Input form for persistent script injection.



14: Stored XSS Execution

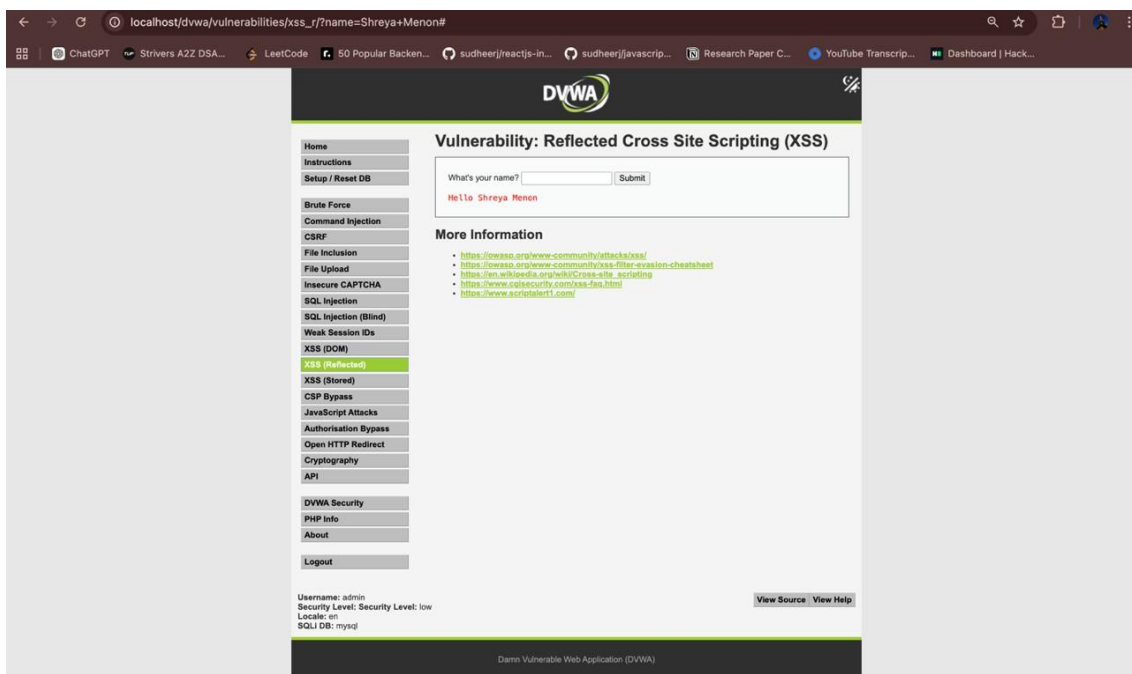
Persistent XSS payload execution via stored input, `<script>alert(1)</script>`



Somaiya Vidyavihar University K. J. Somaiya School of Engineering, Mumbai-77

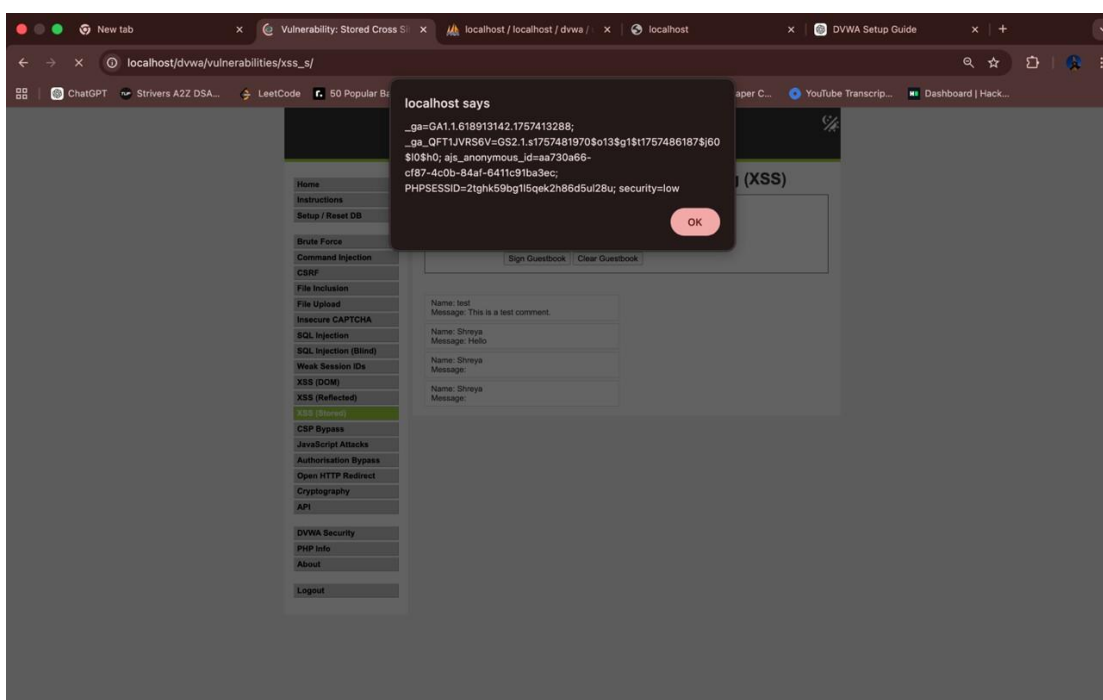
15: Reflected XSS – Normal Input

Reflected XSS page with normal user input.



16: Stored XSS – Cookie Extraction

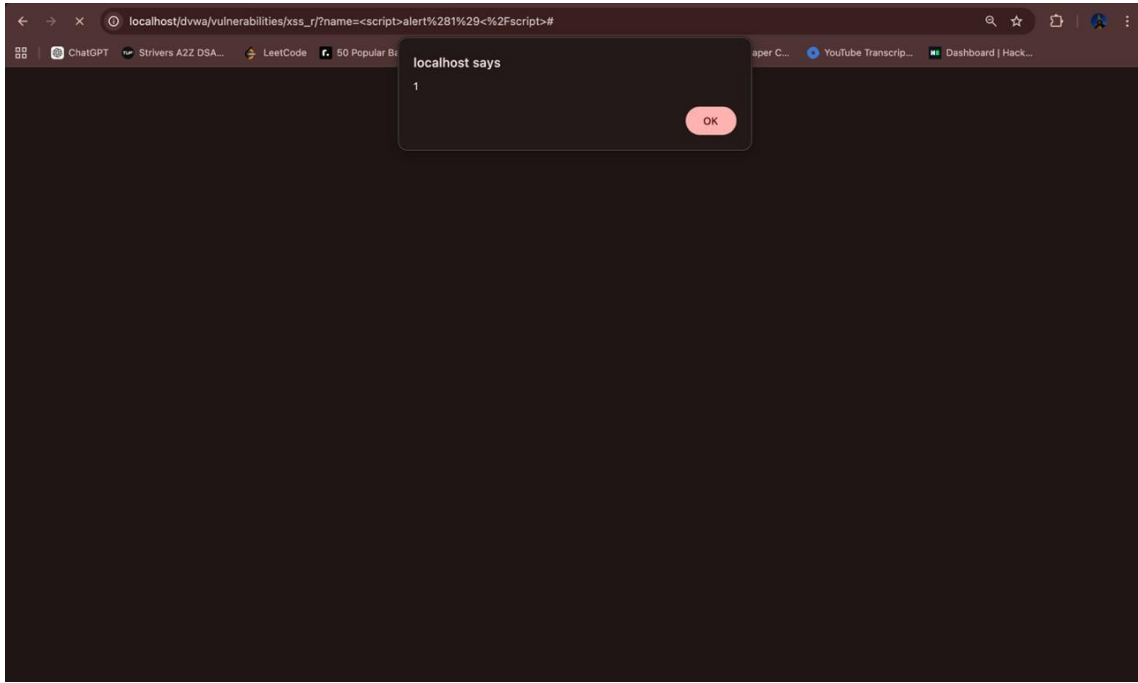
Session cookie theft using XSS payload, `<script>alert(document.cookie)</script>`



Somaiya Vidyavihar University K. J. Somaiya School of Engineering, Mumbai-77

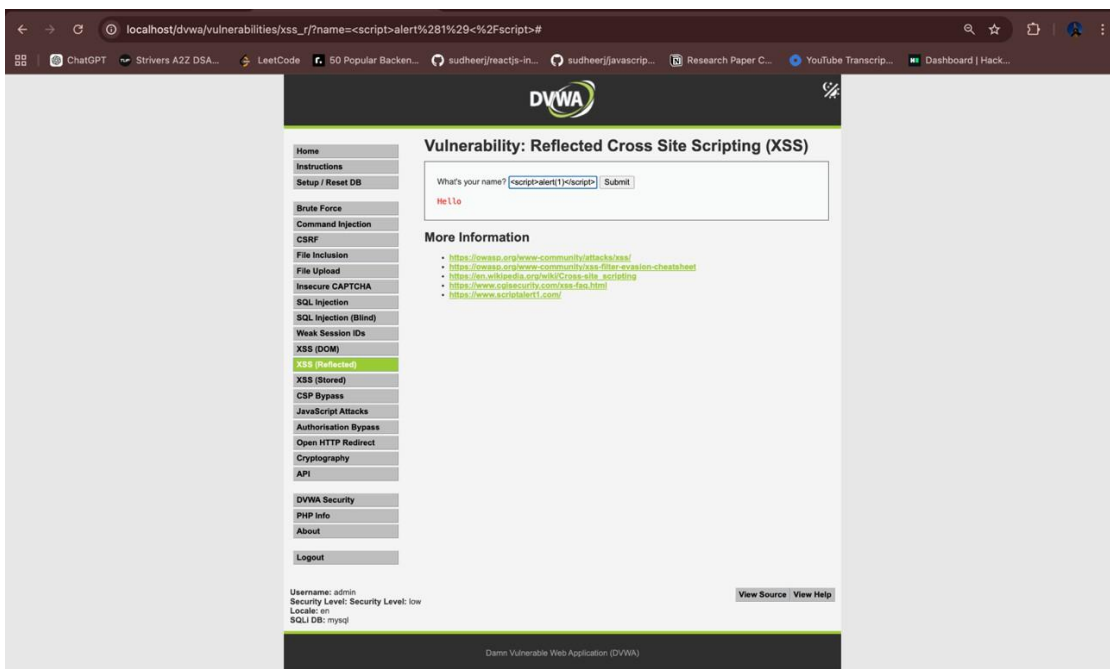
17: Reflected XSS – Alert Execution

Label: Reflected XSS payload executed via URL parameter, `<script>alert(1)</script>`



18: Reflected XSS – Script Injection

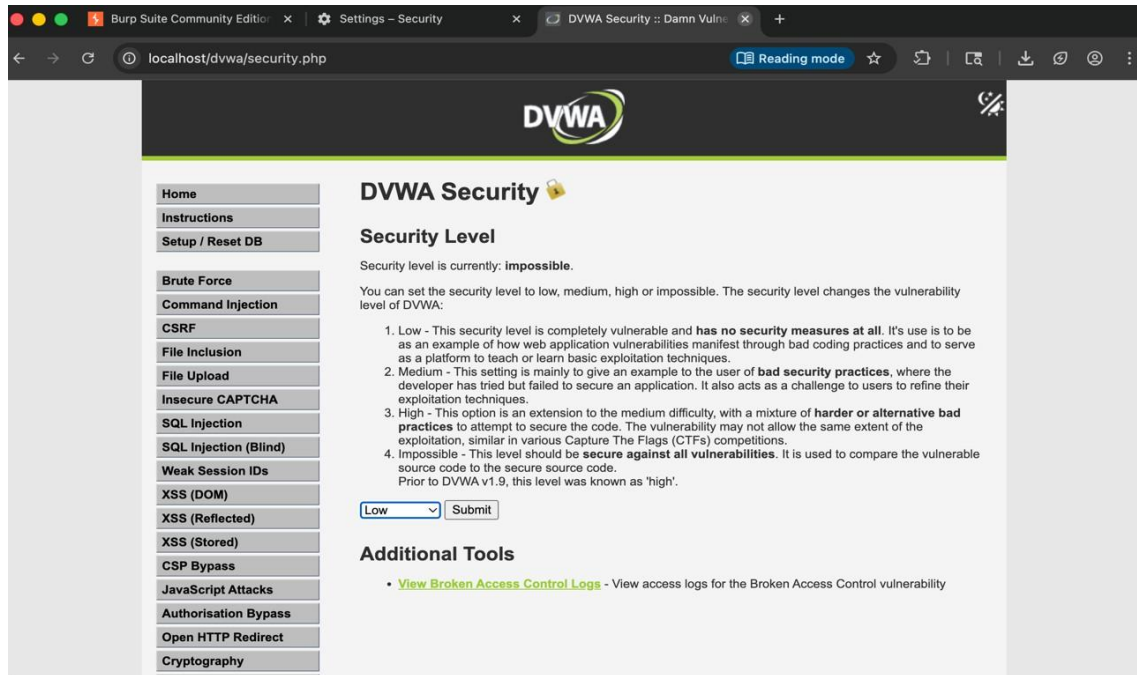
Label: Injected JavaScript payload displayed in input field, `<script>alert(1)</script>`



Somaiya Vidyavihar University K. J. Somaiya School of Engineering, Mumbai-77

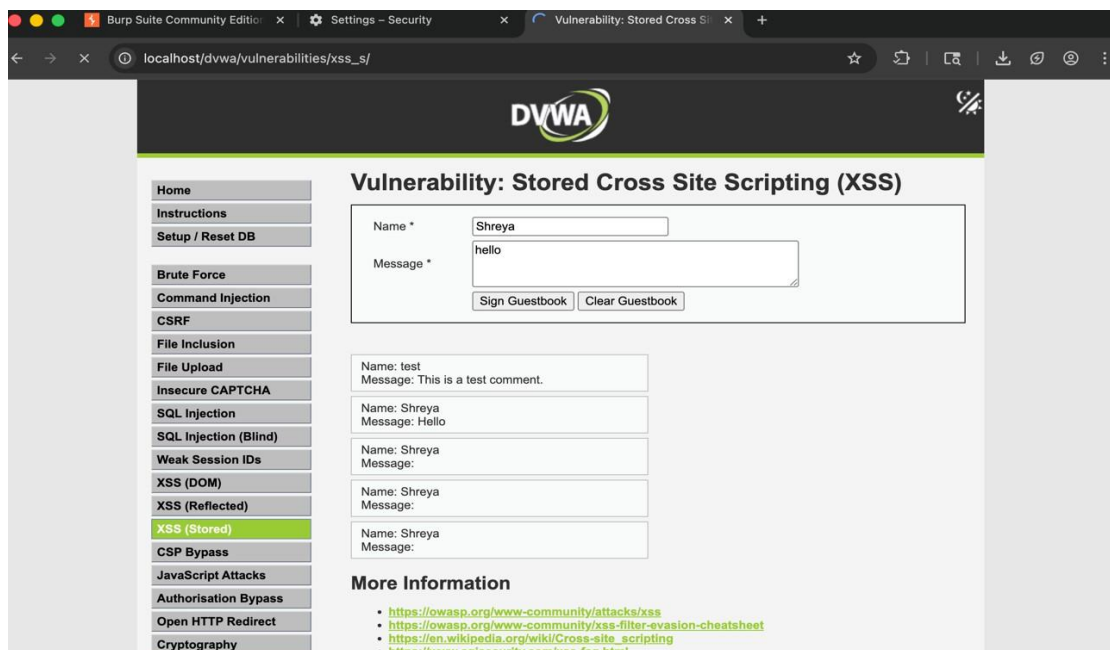
1: DVWA Security Level Configuration

DVWA Security Settings – Setting application security level to *low* for vulnerability testing.



2: Stored XSS Input Form

Stored XSS Vulnerable Input Page – User input fields for persistent script injection.

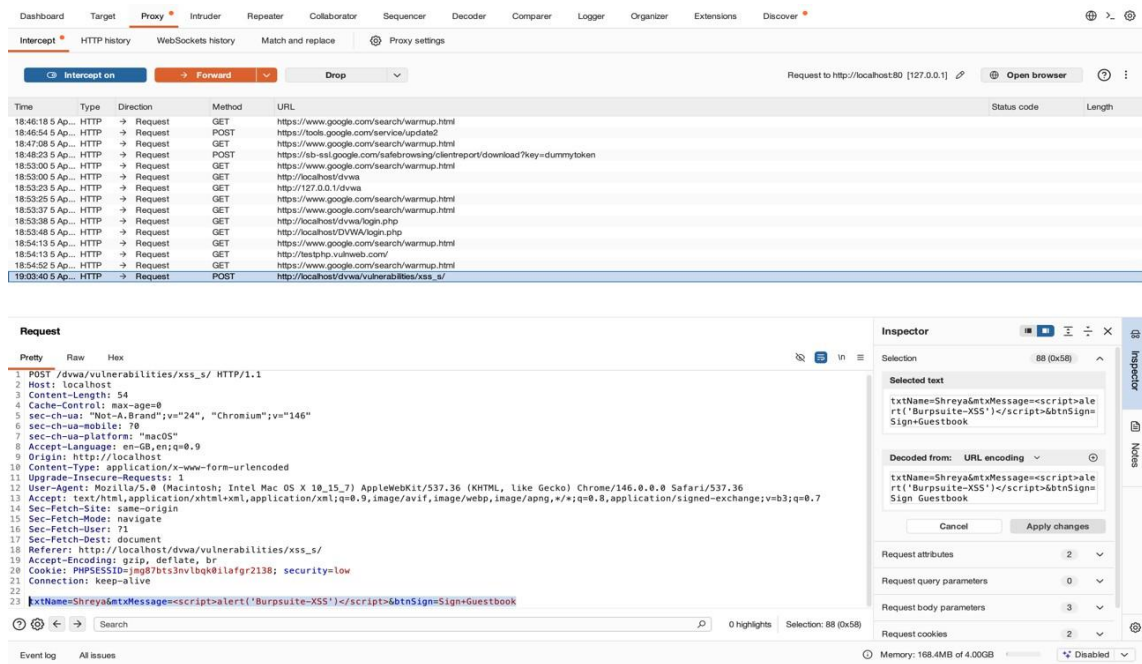


Somaiya Vidyavihar University K. J. Somaiya School of Engineering, Mumbai-77

3: Burp Suite Intercept Request (XSS Payload)

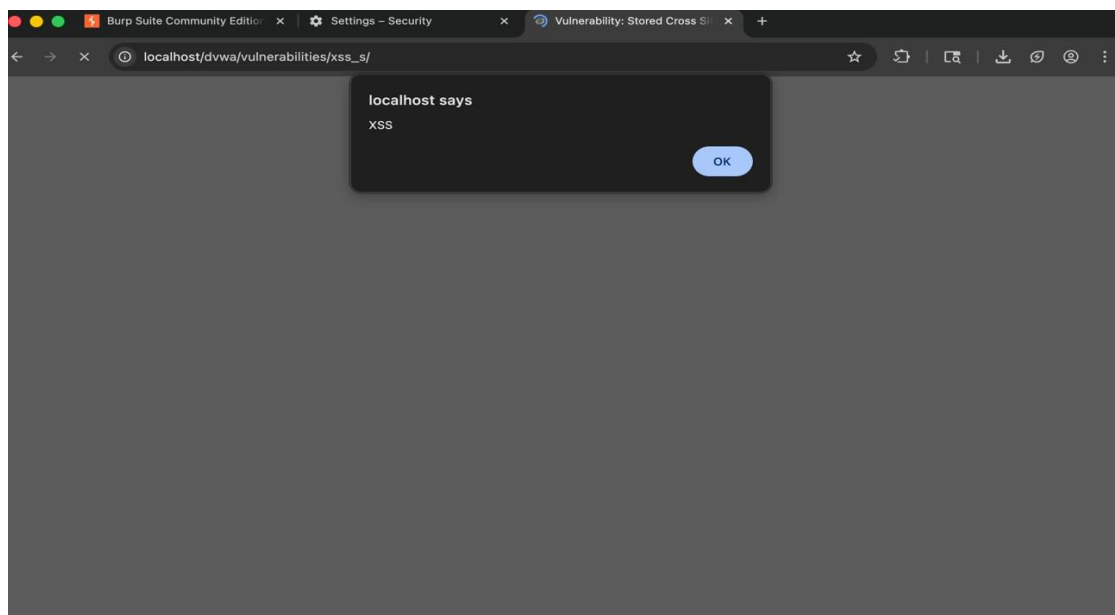
Intercepted HTTP Request containing injected XSS payload via Burp Suite.

`<script>alert('Burpsuite-XSS')</script>`



4: Stored XSS Alert Execution

Successful Stored XSS Execution – JavaScript alert triggered from stored payload.



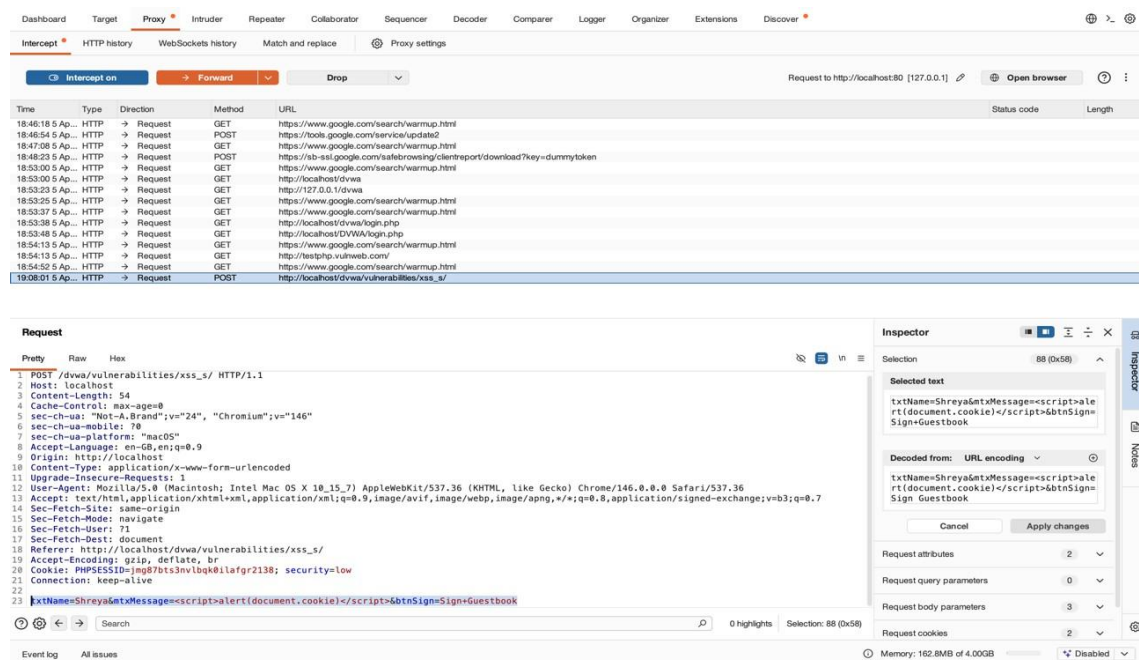
Somaiya Vidyavihar University

K. J. Somaiya School of Engineering, Mumbai-77

5: Burp Suite – Cookie Stealing Payload

Modified HTTP request injecting cookie extraction payload.

`<script>alert(document.cookie)</script>`



The screenshot shows the Burp Suite interface. The top menu bar includes Dashboard, Target, Proxy, Intruder, Repeater, Collaborator, Sequencer, Decoder, Comparer, Logger, Organizer, Extensions, and Discover. The main toolbar has buttons for Intercept on, Forward, and Drop. Below this is a table of intercepted requests. The selected request is a POST to `http://localhost:8080/vulnerabilities/xss_s/`. The request details are shown in the 'Request' tab, and the 'Inspector' tab shows the raw request body with the injected payload `<script>alert(document.cookie)</script>`.

Time	Type	Direction	Method	URL	Status code	Length
18:46:18.5	Ap...	HTTP	→ Request	GET	https://www.google.com/search/warmup.html	
18:46:54.5	Ap...	HTTP	→ Request	POST	https://tools.google.com/service/update2	
18:47:08.5	Ap...	HTTP	→ Request	GET	https://www.google.com/search/warmup.html	
18:48:23.5	Ap...	HTTP	→ Request	POST	https://lbt-sst.google.com/safebrowsing/clientreport/download?key=dummytoken	
18:53:00.5	Ap...	HTTP	→ Request	GET	https://www.google.com/search/warmup.html	
18:53:23.5	Ap...	HTTP	→ Request	GET	http://localhost/dvwa	
18:53:25.5	Ap...	HTTP	→ Request	GET	https://www.google.com/search/warmup.html	
18:53:37.5	Ap...	HTTP	→ Request	GET	https://www.google.com/search/warmup.html	
18:53:38.5	Ap...	HTTP	→ Request	GET	http://localhost/dvwa/login.php	
18:53:48.5	Ap...	HTTP	→ Request	GET	http://localhost/dvwa/login.php	
18:54:13.5	Ap...	HTTP	→ Request	GET	https://www.google.com/search/warmup.html	
18:54:13.5	Ap...	HTTP	→ Request	GET	http://neephp.vulnweb.com/	
18:54:52.5	Ap...	HTTP	→ Request	GET	https://www.google.com/search/warmup.html	
19:08:01.5	Ap...	HTTP	→ Request	POST	http://localhost/dvwa/vulnerabilities/xss_s/	

Request

Pretty Raw Hex

```
1 POST /dvwa/vulnerabilities/xss_s/ HTTP/1.1
2 Host: localhost
3 Content-Length: 54
4 Cache-Control: max-age=0
5 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "macOS"
8 Accept-Language: en-GB,en;q=0.9
9 Origin: http://localhost
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0 Safari/537.36
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://localhost/dvwa/vulnerabilities/xss_s/
19 Accept-Encoding: gzip, deflate, br
20 Cookie: PHPSESSID=jmg87bts3nvlbqk0llafr2138; security=low
21 Connection: keep-alive
22
23 txtName=Shreya&txtMessage=<script>alert(document.cookie)</script>&btnSign=Sign+Guestbook
```

Inspector

Selection: 88 (0x58)

Selected text

```
txtName=Shreya&txtMessage=<script>alert(document.cookie)</script>&btnSign=Sign+Guestbook
```

Decoded from: URL encoding

Cancel Apply changes

Request attributes: 2

Request query parameters: 0

Request body parameters: 3

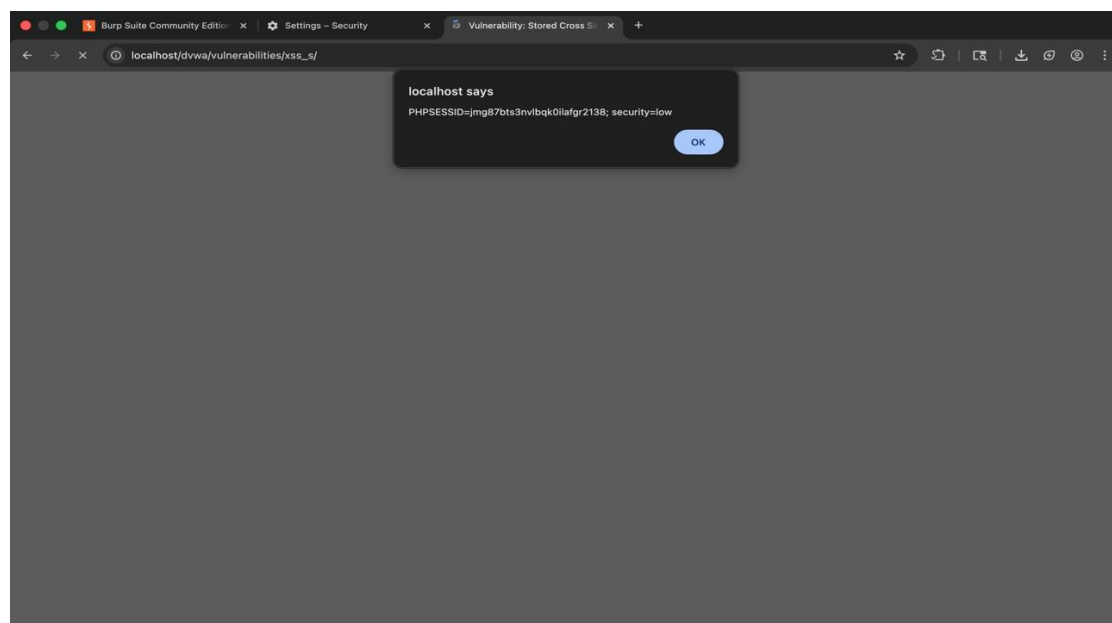
Request cookies: 2

Memory: 162.8MB of 4.00GB

Event log All issues

6: Cookie Disclosure via XSS

Session Cookie Exposure using XSS vulnerability.



The screenshot shows a web browser window with the address bar displaying `localhost/dvwa/vulnerabilities/xss_s/`. A modal dialog box is open, displaying the message "localhost says" and the cookie value `PHPSESSID=jmg87bts3nvlbqk0llafr2138; security=low`. The dialog has an "OK" button.

Somaiya Vidyavihar University K. J. Somaiya School of Engineering, Mumbai-77

7: Burp Suite – Normal Input Interception

Intercepted HTTP request with normal input for comparison.

hello+phishing

The screenshot shows the Burp Suite interface with the 'Intercept' tab selected. A list of intercepted HTTP requests is displayed. The selected request is a POST to `http://localhost:8080/vulnerabilities/xss_s/`. The 'Request' tab shows the raw HTTP request, and the 'Inspector' tab shows the request details.

Request

```

1 POST /dwa/vulnerabilities/xss_s/ HTTP/1.1
2 Host: localhost
3 Content-Length: 63
4 Cache-Control: max-age=0
5 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "macOS"
8 Accept-Language: en-GB,en;q=0.9
9 Origin: http://localhost
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://localhost/dwa/vulnerabilities/xss_s/
19 Accept-Encoding: gzip, deflate, br
20 Cookie: PHPSESSID=jmg87bts3nvlbqk0llafr2138; security=low
21 Connection: keep-alive
22
23 txtName=Shreya&txtMessage=hello+phishing&btnSign=Sign+Guestbook
  
```

Inspector

- Request attributes: 2
- Request query parameters: 0
- Request body parameters: 3
- Request cookies: 2
- Request headers: 20

8: Session Expired Page

Session Handling Behavior after manipulated request.

The screenshot shows a web browser window with the address `http://localhost/dwa/vulnerabilities/xss_s/`. The page displays a message: "Session Expired - Please Login Again". Below the message is a login form with the following fields:

- Username:
- Password:
- Login button

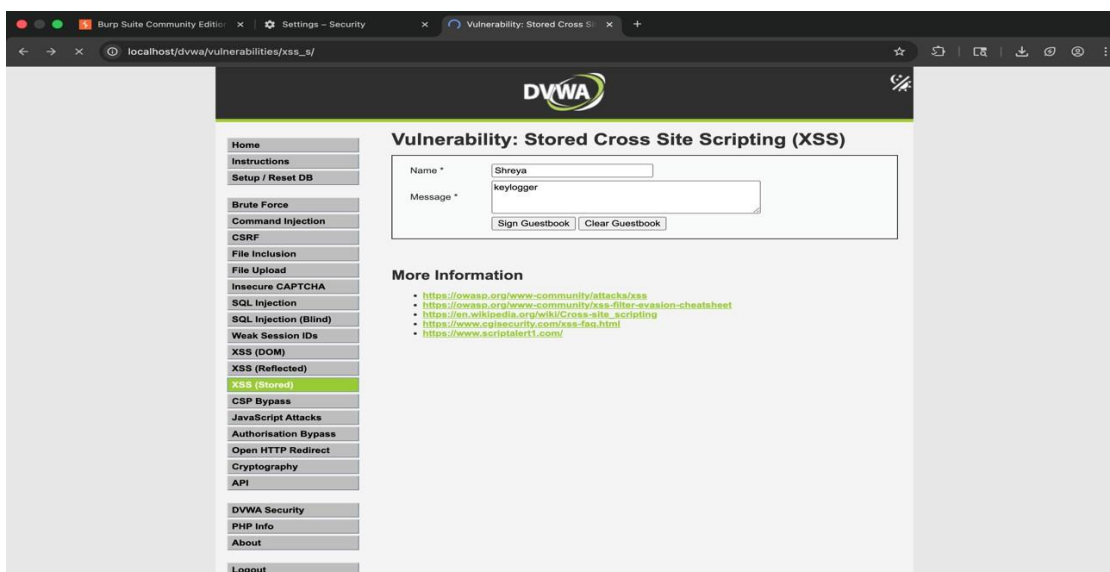
Somaiya Vidyavihar University K. J. Somaiya School of Engineering, Mumbai-77

9: Stored XSS – Additional Payload Input

Injecting advanced JavaScript payload via Burp Suite.

```

<script>
document.onkeypress = function(e){
    alert("Key pressed: " + e.key); }
</script>
  
```

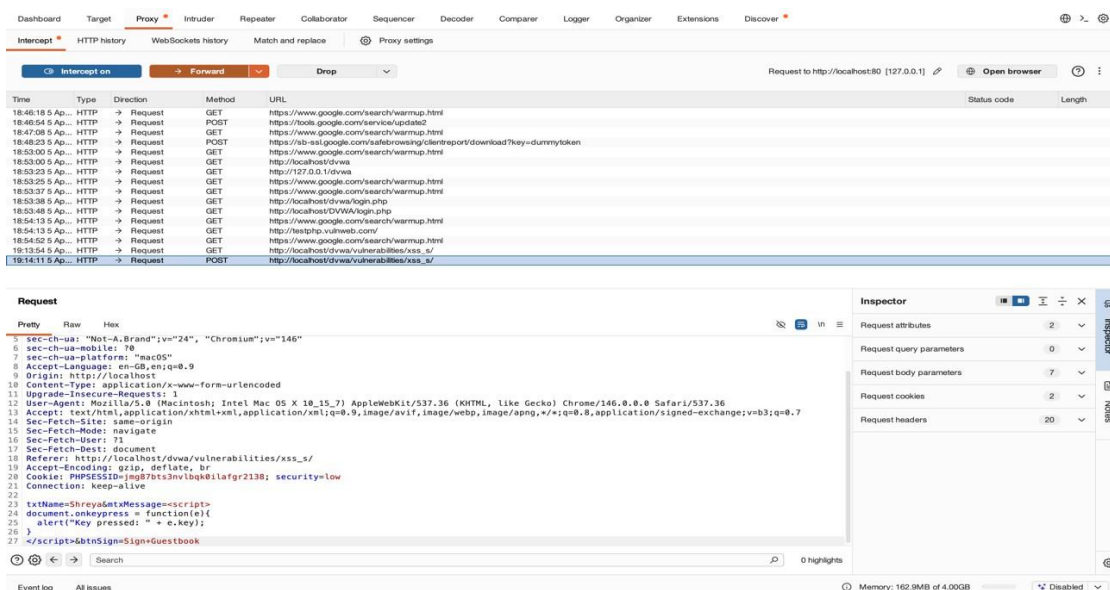


10: Burp Suite – DOM Manipulation Payload

Injecting DOM manipulation script through intercepted request.

```

<script>alert(document.body.innerHTML)</script>
  
```



Somaiya Vidyavihar University K. J. Somaiya School of Engineering, Mumbai-77

11: Stored XSS – HTML Injection Output

Display of injected HTML content via XSS vulnerability.

The screenshot shows the Burp Suite interface with the 'Intercept' tab selected. A list of intercepted HTTP requests is displayed, including various GET and POST requests to localhost and external URLs. The selected request is a POST to 'http://localhost/dvwa/vulnerabilities/xss_s/'. The 'Request' tab is active, showing the raw HTTP data. The 'Inspector' tab on the right shows the request body parameters, including 'PHPSESSID' and 'security'.

The screenshot shows a web browser window displaying the output of a stored XSS attack. The browser address bar shows 'localhost/dvwa/vulnerabilities/xss_s/'. A modal dialog box titled 'localhost says' is displayed, showing the injected HTML content. The content includes a navigation menu with links to 'Home', 'Instructions', 'Setup / Reset DB', 'Brute Force', 'Command Injection', and 'CSRF'.

Somaiya Vidyavihar University
K. J. Somaiya School of Engineering, Mumbai-77

Conclusion:

This experiment provided an understanding of critical web vulnerabilities such as XSS and SQL Injection. By exploring these attacks in a controlled environment using DVWA and Burp Suite, we observed how insecure input handling can lead to severe security breaches. The lab reinforced the importance of secure coding practices, proper validation, and defensive mechanisms to protect web applications from such attacks.

Post-Lab Questions:

1. What is XSS and how does it differ from SQL Injection?

XSS targets the client-side by injecting malicious scripts into web pages, while SQL Injection targets the server-side by manipulating database queries.

2. What are the different types of XSS?

Stored, Reflected, and DOM-based XSS.

3. How can XSS be prevented?

By input validation, output encoding, CSP, and secure cookie handling.

4. What is SQL Injection?

An attack where malicious SQL queries are injected into input fields to manipulate databases.

5. What are prepared statements?

They separate SQL logic from user input, preventing injection attacks.

6. What is DVWA used for?

It is used for practicing and understanding web vulnerabilities in a safe environment.

7. What role does Burp Suite play?

It helps intercept, analyze, and modify HTTP requests to identify vulnerabilities.